returned something like `$ads['registration'] === $reps['registration']` the callback would have been indicating that the second element was always greater than first (and the intersect would have been empty).

For checking keys, the callback works in much the same way: it must function correctly for sorting as well as comparison. Let's say that the two databases don't just have different data models, they also have different key values. This causes us a problem with `array_uintersect` because the callback for value comparison is also a callback for sorting, which means it doesn't just compare values between `$advertised` and `$outForRepair` it also compares values in each array against each other. Let's make a small change to `$outForRepair` and run it through `array_uintersect()` as before:

```
// ... $advertised as before

$outForRepair = [
    [
        'x_registration' => 'AAA 1A',
        'x_fault' => 'Missing towel',
    ]
];

$deListUntilRepaired = array_uintersect($advertised, $outForRepair,
function ($ads, $reps) {
    return $ads['registration'] <=> $reps['x_registration'];
});
print_r($deListUntilRepaired);

Output:
Warning: Undefined array key "x_registration"...
Array
(
)
```

There are two problems here. First, when the callback is used to sort `$advertised` we get a warning, because there is no `x_registration` key in the sub-arrays of `$advertised`. Second, and more subtle, is that the computed intersect is unreliable because internal sorting is broken. When `$advertised` is sorted we're testing a key string (which will always have a value) against an undefined value, and the result is always `1`: PHP is being told that every element is greater than every other element, this has the effect of reducing